

**LAPORAN UJIAN TENGAH SEMESTER
PRAKTIKUM PEMROGRAMAN BACKEND LANJUT**



Disusun Oleh :

Alifia Putri Marizka 434241103

Repository :

<https://github.com/Lifiuuu/SIAKAD-MINI.git>

Program Studi D4 Teknik Informatika

Fakultas Vokasi

Universitas Airlangga

Tahun Ajaran 2026/2027

DAFTAR ISI

DAFTAR ISI.....	2
BAB I.....	4
PENDAHULUAN.....	4
1.1 Latar Belakang.....	4
1.2 Teknologi yang Digunakan.....	4
1.3 Struktur Direktori Proyek.....	4
BAB II.....	5
STRUKTUR DATABASE DAN ENTITAS.....	5
2.1 Migration dan Seeder Database.....	5
2.2 Tabel dan Relasi (Users, Students, Courses, Enrollments).....	8
BAB III.....	9
PEMBAHASAN LOGIKA BISNIS DAN KODE KRUSIAL.....	9
3.1 Autentikasi JWT (JSON Web Token).....	9
3.2 Role-Based Access Control (RBAC).....	9
3.3 Aturan Bisnis KRS, Penanganan Transaksi dan Row-Level Locking.....	10
3.4 Custom Validator (NIM, Angkatan, Tahun Akademik).....	11
3.5 Proteksi Brute-Force (Rate Limiter) dan Keamanan CORS.....	12
3.6 Algoritma Pagination (Offset/Limit) dan Meta Format.....	13
3.7 Implementasi Soft Delete.....	14
3.8 Centralized Error Handling dan Hashing (Bcrypt).....	15
BAB IV.....	16
IMPLEMENTASI DAN PENGUJIAN 10 ENDPOINT TERPADU (UAT).....	16
4.1 Standarisasi Format Response JSON dan Ringkasan 10 Endpoint.....	16
4.2 Tabel Ringkasan Matriks Skenario Pengujian (UAT).....	17
4.3 Rincian Spesifikasi dan Bukti Pengujian per Endpoint (Positive dan Negative Test). 20	
4.3.1 Endpoint 1: POST /api/v1/auth/login.....	20
4.3.2 Endpoint 2: GET /api/v1/auth/me.....	22
4.3.3 Endpoint 3: GET /api/v1/students.....	23
4.3.4 Endpoint 4: POST /api/v1/students.....	25
4.3.5 Endpoint 5: GET /api/v1/students/{id}.....	26
4.3.6 Endpoint 6: PUT /api/v1/students/{id}.....	28

4.3.7 Endpoint 7: DELETE /api/v1/students/{id}.....	29
4.3.8 Endpoint 8: GET /api/v1/courses.....	31
4.3.9 Endpoint 9: POST /api/v1/enrollments.....	32
4.3.10 Endpoint 10: DELETE /api/v1/enrollments/{id}.....	35
BAB V.....	38
KESIMPULAN DAN SARAN PENGEMBANGAN.....	38
5.1 Kesimpulan.....	38

BAB I

PENDAHULUAN

1.1 Latar Belakang

SIAKAD Mini adalah sebuah layanan akademik berbasis RESTful API yang dirancang secara kokoh untuk menangani operasi manajemen kampus, khususnya pengelolaan data mahasiswa, mata kuliah, dan Kartu Rencana Studi (KRS). Sistem ini difokuskan pada integritas data yang ketat melalui validasi input, autentikasi token terpusat pada semua endpoint (kecuali login), otorisasi berbasis peran (RBAC), serta keamanan transaksi data relasional.

API ini melayani dua peran pengguna utama:

1. Admin: Mengelola data mahasiswa dan mata kuliah.
2. Mahasiswa: Melihat profil sendiri, serta mengambil dan membatalkan mata kuliah pada KRS.

1.2 Teknologi yang Digunakan

1. Bahasa Pemrograman: Golang (Go)
2. Web Framework: Go Fiber (gofiber/fiber/v2)
3. Relational Database: PostgreSQL dengan Driver pgx (jackc/pgx/v5)
4. Keamanan: JWT (golang-jwt/jwt/v5), Bcrypt (golang.org/x/crypto/bcrypt), Fiber Helmet, Fiber CORS, dan Fiber Rate Limiter.

1.3 Struktur Direktori Proyek

Proyek ini menerapkan konsep arsitektur modular (Clean Architecture / Layered Architecture):

1. app/model/: Struktur entitas data, representasi JSON, dan tag validasi.
2. app/repository/: Lapisan akses data untuk eksekusi SQL native ke PostgreSQL.
3. app/service/: Business Logic Layer (validasi batas SKS berdasarkan IPK, pengecekan kuota, dan manajemen transaksi).
4. config/: Konfigurasi koneksi database, variabel environment (.env), dan logger.
5. helper/: Custom Validator, JWT generator/parser, dan Centralized Response Builder.
6. middleware/: Interceptor HTTP (Autentikasi JWT, RBAC, Rate Limiter, Helmet, CORS).
7. migrations/: Skrip DDL pembuatan tabel dan seeder data awal.
8. route/: Pendaftaran seluruh endpoint URL API beserta grup middleware-nya.

BAB II

STRUKTUR DATABASE DAN ENTITAS

2.1 Migration dan Seeder Database

Database dibuat secara sekuensial menggunakan skrip SQL murni di folder /migrations. Sesuai ketentuan wajib spesifikasi UTS, proses seeder secara otomatis meng-inject data awal yang terdiri dari:

1. Minimal 1 akun Admin (beserta password yang telah di-hash menggunakan Bcrypt).
2. Minimal 20 data Mahasiswa (terhubung 1-to-1 dengan akun pada tabel users ber-role mahasiswa).
3. Minimal 10 data Mata Kuliah dengan variasi SKS, semester, dan kuota kelas.

Bukti Potongan Kode Migration & Seeder Database:

```
❏ SIAKAD_MINI > migrations > 001_create_users.sql
1 CREATE TABLE IF NOT EXISTS users (
2   id SERIAL PRIMARY KEY,
3   email VARCHAR(255) NOT NULL,
4   password VARCHAR(255) NOT NULL,
5   role VARCHAR(20) NOT NULL DEFAULT 'mahasiswa',
6   created_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
7 );
8
9 CREATE UNIQUE INDEX IF NOT EXISTS users_email_lower_key
10 ON users (LOWER(email));
11
```

```
❏ SIAKAD_MINI > migrations > 002_create_students.sql
1 CREATE TABLE IF NOT EXISTS students (
2   id SERIAL PRIMARY KEY,
3   user_id INTEGER NOT NULL REFERENCES users(id) ON UPDATE CASCADE ON DELETE CASCADE,
4   nim VARCHAR(12) NOT NULL,
5   nama VARCHAR(255) NOT NULL,
6   prodi VARCHAR(100) NOT NULL,
7   angkatan SMALLINT NOT NULL,
8   ipk_terakhir DOUBLE PRECISION NOT NULL DEFAULT 0,
9   deleted_at TIMESTAMPTZ,
10  created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
11  UNIQUE(user_id) -- 1-1 relationship with users
12 );
13
14 CREATE UNIQUE INDEX IF NOT EXISTS students_nim_key
15 ON students (nim);
16
17 CREATE INDEX IF NOT EXISTS students_nama_lower_idx
18 ON students (LOWER(nama));
19
20 CREATE INDEX IF NOT EXISTS students_created_at_id_desc_idx
21 ON students (created_at DESC, id DESC);
22
```

```

❏ SIAKAD_MINI > migrations > 003_create_courses.sql
1 CREATE TABLE IF NOT EXISTS courses (
2     id SERIAL PRIMARY KEY,
3     kode_mk VARCHAR(20) NOT NULL,
4     nama_mk VARCHAR(255) NOT NULL,
5     sks SMALLINT NOT NULL DEFAULT 2,
6     semester SMALLINT NOT NULL DEFAULT 1,
7     kuota INTEGER NOT NULL DEFAULT 40,
8     created_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
9 );
10
11 CREATE UNIQUE INDEX IF NOT EXISTS courses_kode_mk_lower_key
12     ON courses (LOWER(kode_mk));
13
14 CREATE INDEX IF NOT EXISTS courses_nama_mk_lower_idx
15     ON courses (LOWER(nama_mk));
16

```

```

❏ SIAKAD_MINI > migrations > 004_create_enrollments.sql
1 CREATE TABLE IF NOT EXISTS enrollments (
2     id SERIAL PRIMARY KEY,
3     student_id INTEGER NOT NULL REFERENCES students(id) ON DELETE CASCADE,
4     course_id INTEGER NOT NULL REFERENCES courses(id) ON DELETE CASCADE,
5     tahun_akademik VARCHAR(20) NOT NULL, -- e.g. "2026/2027-Ganjil"
6     created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
7     -- Mahasiswa tidak dapat mengambil mata kuliah yang sama dua kali pada tahun
8     UNIQUE (student_id, course_id, tahun_akademik)
9 );
10
11 CREATE INDEX IF NOT EXISTS enrollments_student_id_idx
12     ON enrollments (student_id);
13
14 CREATE INDEX IF NOT EXISTS enrollments_course_id_idx
15     ON enrollments (course_id);
16

```

```

❏ SIAKAD_MINI > migrations > 006_rbac.sql
1 -----
2 -- roles – daftar role yang diakui sistem
3 -----
4 CREATE TABLE IF NOT EXISTS roles (
5     name VARCHAR(20) PRIMARY KEY,
6     description VARCHAR(150) NOT NULL,
7     created_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
8 );
9
10 INSERT INTO roles (name, description) VALUES
11     ('admin', 'Akses penuh terhadap seluruh data dan pengaturan'),
12     ('mahasiswa', 'Hanya boleh melihat profil, mengambil dan membatalkan mata kuliah')
13 ON CONFLICT (name) DO NOTHING;
14
15 -----
16 -- permissions – daftar tindakan yang dapat diberikan kepada role
17 -----
18 CREATE TABLE IF NOT EXISTS permissions (
19     name VARCHAR(50) PRIMARY KEY,
20     description VARCHAR(150) NOT NULL
21 );
22
23 INSERT INTO permissions (name, description) VALUES
24     -- Students permissions
25     ('students:list', 'Melihat daftar seluruh mahasiswa'),
26     ('students:read:any', 'Melihat data mahasiswa mana pun'),
27     ('students:create', 'Menambahkan data mahasiswa baru');

```

```

INSERT INTO permissions (name, description) VALUES
-- Students permissions
('students:list', 'Melihat daftar seluruh mahasiswa'),
('students:read:any', 'Melihat data mahasiswa mana pun'),
('students:create', 'Menambahkan data mahasiswa baru'),
('students:update:any', 'Mengubah data mahasiswa mana pun'),
('students:delete', 'Menghapus data mahasiswa'),
-- Courses permissions
('courses:list', 'Melihat daftar mata kuliah'),
('courses:create', 'Menambahkan mata kuliah baru'),
('courses:update:any', 'Mengubah data mata kuliah mana pun'),
('courses:delete', 'Menghapus mata kuliah'),
-- Enrollments permissions
('enrollments:list', 'Melihat daftar krs'),
('enrollments:read:any', 'Melihat krs mahasiswa mana pun'),
('enrollments:create', 'Menambahkan krs'),
('enrollments:delete', 'Menghapus krs')
ON CONFLICT (name) DO NOTHING;

```

```

42
43  -----
44  -- role_permissions - tabel penghubung, inti dari model RBAC
45  -----
46  CREATE TABLE IF NOT EXISTS role_permissions (
47      role_name VARCHAR(20) NOT NULL
48      REFERENCES roles(name) ON DELETE CASCADE,
49      permission_name VARCHAR(50) NOT NULL
50      REFERENCES permissions(name) ON DELETE CASCADE,
51      PRIMARY KEY (role_name, permission_name)
52  );
53
54  INSERT INTO role_permissions (role_name, permission_name) VALUES
55      -- Admin: akses penuh
56      ('admin', 'students:list'),
57      ('admin', 'students:read:any'),
58      ('admin', 'students:create'),
59      ('admin', 'students:update:any'),
60      ('admin', 'students:delete'),
61      ('admin', 'courses:list'),
62      ('admin', 'courses:create'),
63      ('admin', 'courses:update:any'),
64      ('admin', 'courses:delete'),
65      -- Mahasiswa
66      ('mahasiswa', 'courses:list'),
67      ('mahasiswa', 'enrollments:list'),
68      ('mahasiswa', 'enrollments:create'),
69      ('mahasiswa', 'enrollments:delete')
ON CONFLICT DO NOTHING;

```

```

-- Kunci column role pada users agar hanya berisi role yang dikenal.
UPDATE users SET role = 'mahasiswa' WHERE role NOT IN (SELECT name FROM roles);

ALTER TABLE users DROP CONSTRAINT IF EXISTS users_role_fkey;
ALTER TABLE users
    ADD CONSTRAINT users_role_fkey
    FOREIGN KEY (role) REFERENCES roles(name) ON UPDATE CASCADE;

CREATE INDEX IF NOT EXISTS users_role_idx ON users (role);

-- Cursor index untuk user
CREATE INDEX IF NOT EXISTS users_created_at_id_desc_idx
    ON users (created_at DESC, id DESC);

```

```
❏ SIAKAD_MINI > migrations > 007_seeder.sql
1  -- Seeder untuk SIAKAD Mini
2  -- Menambahkan 1 admin, 20 mahasiswa, dan 10 mata kuliah.
3
4  -- 1. Insert 1 Admin
5  INSERT INTO users (email, password, role) VALUES
6  ('admin@siakad.ac.id', '$2a$12$Y/B1rE6zP/G3H7X0Z.07n.4y6c0c.q/Z2M3D.y1w6x7X9w9U/T0K.', 'admin'
7  ON CONFLICT DO NOTHING;
8  Ctrl+L for Agent
9  -- 2. Insert 20 Mahasiswa
10 DO $$
11 DECLARE
12     i INT;
13     uid INT;
14 BEGIN
15     FOR i IN 1..20 LOOP
16         INSERT INTO users (email, password, role)
17         VALUES ('student' || i || '@siakad.ac.id', '$2a$12$Y/B1rE6zP/G3H7X0Z.07n.4y6c0c.q/Z2M3
18         RETURNING id INTO uid;
19
20         INSERT INTO students (user_id, nim, nama, prodi, angkatan, ipk_terakhir)
21         VALUES (
22             uid,
23             '18722100' || LPAD(i::text, 4, '0'),
24             'Mahasiswa ' || i,
25             'Informatika',
26             2026,
27             (random() * (4.0 - 2.0) + 2.0) -- Random IPK between 2.0 and 4.0
28     );
29     END LOOP;
30 END;
```

2.2 Tabel dan Relasi (Users, Students, Courses, Enrollments)

Berikut adalah spesifikasi model data minimal beserta relasi antar-entitas yang diterapkan di PostgreSQL:

Tabel	Kolom Utama	Relasi dan Constraint
users	id, email, password, role (admin/mahasiswa)	Relasi 1-to-1 ke tabel students. Kolom email bersifat unik dan password disimpan dalam bentuk hash.
students	id, user_id, nim (uniq), nama, prodi, angkatan, ipk_terakhir, deleted_at	Berelasi 1-to-1 dengan users (user_id) dan 1-to-N ke enrollments. Terdapat UNIQUE index pada nim serta mendukung soft delete melalui deleted_at.
courses	id, kode_mk (uniq), nama_mk, sks, semester, kuota	Relasi 1-to-N ke tabel enrollments. Terdapat UNIQUE index pada kode_mk.
enrollments	id, student_id, course_id, tahun_akademik, created_at	Tabel pivot KRS. Dilindungi oleh composite UNIQUE constraint pada (student_id, course_id, tahun_akademik).

BAB III

PEMBAHASAN LOGIKA BISNIS DAN KODE KRUSIAL

3.1 Autentikasi JWT (JSON Web Token)

Seluruh endpoint (kecuali POST /api/v1/auth/login) wajib melalui autentikasi token Bearer. Token JWT memuat payload klaim user_id, email, dan role, dihasilkan menggunakan algoritma HMAC SHA-256, dan ditandatangani menggunakan kunci rahasia dari environment variable (JWT_SECRET).

Bukti Potongan Kode Autentikasi JWT:

```
func (m *JWTManager) GenerateAccess(u model.User) (string, error) {
    now := time.Now()
    claims := accessClaims{
        Email: u.Email,
        Role: u.Role,
        RegisteredClaims: jwt.RegisteredClaims{
            Subject: strconv.Itoa(u.ID),
            Issuer: m.issuer,
            IssuedAt: jwt.NewNumericDate(now),
            ExpiresAt: jwt.NewNumericDate(now.Add(m.accessTTL)),
        },
    },
    token := jwt.NewWithClaims(jwt.SigningMethodHS256, claims)
    return token.SignedString(m.secret)
}
```

```
// Parse memeriksa tanda tangan dan masa berlaku token.
func (m *JWTManager) Parse(tokenString string) (model.AuthUser, error) {
    claims := &accessClaims{}
    token, err := jwt.ParseWithClaims(tokenString, claims,
        func(t *jwt.Token) (any, error) {
            // PEMERIKSAAN WAJIB: pastikan algoritmanya memang yang kita pilih.
            if _, ok := t.Method.(*jwt.SigningMethodHMAC); !ok {
                return nil, fmt.Errorf("algoritma tidak diharapkan: %v", t.Header["alg"])
            }
            return m.secret, nil
        },
        jwt.WithIssuer(m.issuer),
        jwt.WithExpirationRequired(),
    )
    if err != nil {
        if errors.Is(err, jwt.ErrTokenExpired) {
            return model.AuthUser{}, ErrExpiredToken
        }
        return model.AuthUser{}, ErrInvalidToken
    }
    if !token.Valid {
        return model.AuthUser{}, ErrInvalidToken
    }
    userID, err := strconv.Atoi(claims.Subject)
    if err != nil {
        return model.AuthUser{}, ErrInvalidToken
    }
    return model.AuthUser{
        UserID: userID,
        Email: claims.Email,
        Role: claims.Role,
    }, nil
}
```

3.2 Role-Based Access Control (RBAC)

Sistem memiliki middleware RequireRole yang membaca klaim role dari token JWT yang sudah terverifikasi. Middleware ini memblokir akses secara instan dengan status HTTP 403 Forbidden apabila peran pengguna tidak sesuai dengan hak akses endpoint. Selain itu,

terdapat validasi kepemilikan data (ownership check) agar mahasiswa hanya dapat mengakses profil dan mengubah KRS miliknya sendiri.

Bukti Potongan Kode Middleware RBAC:

```
2 // RequireRole memeriksa nama role secara langsung.
3 func RequireRole(roles ...string) fiber.Handler {
4     allowed := make(map[string]struct{}, len(roles))
5     for _, role := range roles {
6         allowed[role] = struct{}{}
7     }
8
9     return func(c *fiber.Ctx) error {
10         user, ok := helper.CurrentUser(c)
11         if !ok {
12             return helper.ErrorResponsePayload(c, fiber.StatusUnauthorized, "belum terautentikasi", nil)
13         }
14         if _, granted := allowed[user.Role]; !granted {
15             return helper.ErrorResponsePayload(c, fiber.StatusForbidden, "role Anda tidak berhak mengakses endpoint ini", nil)
16         }
17         return c.Next()
18     }
19 }
```

3.3 Aturan Bisnis KRS, Penanganan Transaksi dan Row-Level Locking

Pengambilan KRS (POST /api/v1/enrollments) menerapkan seluruh Business Rules akademik secara ketat di dalam satu Database Transaction:

1. Perhitungan Batas SKS Berdasarkan IPK Terakhir:
 - a. IPK $\geq 3,00$: Maksimal 24 SKS
 - b. IPK 2,50 – 2,99: Maksimal 21 SKS
 - c. IPK $< 2,50$: Maksimal 18 SKS

Apabila penambahan mata kuliah baru menyebabkan total SKS melampaui batas tersebut, transaksi dibatalkan (rollback) dan mengembalikan error 422 Unprocessable Entity disertai pesan yang menyebutkan sisa SKS yang tersedia.

2. Pencegahan Duplikasi Mata Kuliah:

Mahasiswa tidak dapat mengambil mata kuliah yang sama dua kali pada tahun akademik yang sama. Jika terdeteksi duplikasi, sistem mengembalikan 409 Conflict.

3. Pencegahan Kuota Penuh dan Row-Level Locking:

Sistem menjalankan query SELECT ... FOR UPDATE pada baris mata kuliah yang dituju untuk mengunci baris secara eksklusif selama transaksi berlangsung. Hal ini menjamin kalkulasi sisa_kuota akurat dan bebas dari race condition (overselling) saat banyak mahasiswa mengakses secara bersamaan. Jika sisa_kuota ≤ 0 , sistem menolak permintaan dengan status 422 Unprocessable Entity.

Bukti Potongan Kode Transaksi KRS, Perhitungan Batas SKS, & Row-Level Locking:

```
// 1. Lock the course to safely evaluate quota
var course model.Course
err = tx.QueryRow(ctx,
    `SELECT c.id, c.sks, c.kuota,
        (SELECT COUNT(*) FROM enrollments e WHERE e.course_id = c.id) as terisi
    FROM courses c WHERE c.id = $1 FOR UPDATE`, reqCourse.ID,
).Scan(&course.ID, &course.SKS, &course.Kuota, &course.Terisi)

if err != nil {
    if errors.Is(err, pgx.ErrNoRows) {
        return model.Enrollment{}, ErrNotFound
    }
    return model.Enrollment{}, fmt.Errorf("lock course: %w", err)
}

course.SisaKuota = course.Kuota - course.Terisi
if course.SisaKuota <= 0 {
    return model.Enrollment{}, fmt.Errorf("kuota penuh: %w", ErrConflict)
}
```

```
// 2. Evaluate Max SKS Limits based on IPK
var maxSKS int
if student.IPKTerakhir >= 3.00 {
    maxSKS = 24
} else if student.IPKTerakhir >= 2.50 {
    maxSKS = 21
} else {
    maxSKS = 18
}
```

```
if currentSKS + course.SKS > maxSKS {
    return model.Enrollment{}, fmt.Errorf("batas SKS terlampaui (max: %d, diambil: %d, matkul: %d): %w", maxSKS, currentSKS, course.SKS, ErrConflict)
}
```

```
// 3. Insert Enrollment
var en model.Enrollment
err = tx.QueryRow(ctx,
    `INSERT INTO enrollments (student_id, course_id, tahun_akademik) VALUES ($1, $2, $3) RETURNING id, created_at`,
    student.ID, course.ID, tahunAkademik,
).Scan(&en.ID, &en.CreatedAt)

if err != nil {
    if isUniqueViolation(err) {
        return model.Enrollment{}, fmt.Errorf("sudah diambil: %w", ErrDuplicate)
    }
    return model.Enrollment{}, fmt.Errorf("insert enrollment: %w", err)
}
en.StudentID = student.ID
```

3.4 Custom Validator (NIM, Angkatan, Tahun Akademik)

Validator tingkat lanjut di helper/validator.go memastikan integritas format data sebelum menyentuh lapisan database:

1. NIM: Wajib unik dan tepat 12 digit numerik.
2. Angkatan: Wajib 4 digit numerik dan tidak boleh melebihi tahun berjalan ($\leq$ tahun berjalan).
3. IPK Terakhir: Opsional, bernilai desimal dalam rentang 0,00 hingga 4,00.

4. Tahun Akademik: Mengikuti pola Regex khusus `^\d{4}/\d{4}-(Ganjil|Genap)$` (contoh: 2026/2027-Ganjil).

Bukti Potongan Kode Custom Validator:

```
var tahunAkademikRegex = regexp.MustCompile(`^\d{4}/\d{4}-(Ganjil|Genap)$`)
```

```
//  
- = v.RegisterValidation("nim", func(fl validator.FieldLevel) bool {  
-     val := fl.Field().String()  
-     if len(val) != 12 {  
-         return false  
-     }  
-     for _, r := range val {  
-         if !unicode.IsDigit(r) {  
-             return false  
-         }  
-     }  
-     return true  
- })  
- = v.RegisterValidation("angkatan", func(fl validator.FieldLevel) bool {  
-     val := int(fl.Field().Int())  
-     currentYear := time.Now().Year()  
-     return val >= 1000 && val <= 9999 && val <= currentYear  
- })  
- = v.RegisterValidation("tahun_akademik", func(fl validator.FieldLevel) bool {  
-     return tahunAkademikRegex.MatchString(fl.Field().String())  
- })  
- return v  
}
```

3.5 Proteksi Brute-Force (Rate Limiter) dan Keamanan CORS

Endpoint login dilindungi oleh middleware LoginRateLimiter yang membatasi percobaan login gagal maksimal 5 kali per menit per IP. Jika melewati batas, server langsung mengembalikan status 429 Too Many Requests. API juga dilengkapi middleware Helmet untuk mengamankan HTTP Header dan CORS untuk membatasi akses domain frontend.

Bukti Potongan Kode Rate Limiter & CORS:

```
1 // LoginRateLimiter membatasi jumlah percobaan login dari satu alamat IP.  
2 func LoginRateLimiter() fiber.Handler {  
3     return limiter.New(limiter.Config{  
4         Max: 5,  
5         Expiration: 1 * time.Minute,  
6         KeyGenerator: func(c *fiber.Ctx) string {  
7             return c.IP()  
8         },  
9         LimitReached: func(c *fiber.Ctx) error {  
10            c.Set("Retry-After", "60")  
11            return helper.TooManyRequests("terlalu banyak percobaan login, coba lagi dalam satu menit")  
12        },  
13    })  
14 }  
15
```

```
// Register memasang seluruh middleware yang berlaku untuk semua route.
// URUTAN PENTING: middleware dieksekusi sesuai urutan pemasangan.
func Register(app *fiber.App, logger *slog.Logger, allowedOrigins string) {
    app.Use(requestid.New())
    app.Use(recover.New())
    app.Use(helmet.New())
    app.Use(corsPolicy(allowedOrigins))
    app.Use(RequestLogger(logger))
}

// corsPolicy membatasi origin yang boleh memanggil API.
func corsPolicy(allowedOrigins string) fiber.Handler {
    if strings.TrimSpace(allowedOrigins) == "" {
        allowedOrigins = "http://localhost:5173"
    }
    return cors.New(cors.Config{
        AllowOrigins: allowedOrigins,
        AllowMethods: "GET,POST,PUT,PATCH,DELETE,OPTIONS",
        AllowHeaders: "Origin,Content-Type,Accept,Authorization",
    })
}
```

3.6 Algoritma Pagination (Offset/Limit) dan Meta Format

Endpoint daftar mahasiswa (GET /api/v1/students) menggunakan paginasi SQL LIMIT dan OFFSET (default page = 1, per_page = 10, maksimal 50), mendukung filter prodi dan angkatan, pencarian (search pada nim atau nama), serta pengurutan (sort berdasarkan nama atau -ipk_terakhir). Metadata halaman (current_page, per_page, total, last_page) dihitung secara dinamis dari total baris yang memenuhi kriteria filter.

Bukti Potongan Kode Pagination & Kalkulasi Meta:

```
page, _ := strconv.Atoi(c.Query("page", "1"))
if page < 1 {
    page = 1
}
perPage, _ := strconv.Atoi(c.Query("per_page", "10"))
if perPage < 1 {
    perPage = 10
}
if perPage > 50 {
    perPage = 50
}
```

```

lastPage := total / perPage
if total%perPage > 0 {
    lastPage++
}
if lastPage == 0 {
    lastPage = 1
}

meta := model.Meta{
    CurrentPage: page,
    PerPage:     perPage,
    Total:       total,
    LastPage:    lastPage,
}

```

```

// Offset menghitung berapa baris yang dilewati untuk halaman ini.
func (q ListQuery) Offset() int {
    return (q.Page - 1) * q.Limit
}

```

3.7 Implementasi Soft Delete

Penghapusan mahasiswa (DELETE /api/v1/students/{id}) dilakukan secara soft delete dengan mengisi timestamp pada kolom deleted_at. Mahasiswa yang telah di-soft delete:

1. Tidak akan muncul pada daftar mahasiswa (GET /api/v1/students).
2. Mengembalikan 404 Not Found jika diakses melalui GET /api/v1/students/{id}.
3. Diblokir saat mencoba melakukan login (401 Unauthorized).
4. Tetap menjaga keutuhan referensi Foreign Key pada riwayat tabel enrollments.

Bukti Potongan Kode Query Soft Delete:

```

func (r *studentPostgresRepository) SoftDelete(ctx context.Context, id int) error {
    tag, err := r.pool.Exec(ctx, "UPDATE students SET deleted_at = NOW() WHERE id = $1 AND deleted_at IS NULL", id)
    if err != nil {
        return fmt.Errorf("soft delete student: %w", err)
    }
    if tag.RowsAffected() == 0 {
        return ErrNotFound
    }
    return nil
}

```

```

func buildStudentFilter(q model.ListQuery) (string, []any) {
    where := " WHERE deleted_at IS NULL"
    args := []any{}
}

```

```
func (r *studentPostgresRepository) FindByID(ctx context.Context, id int) (model.Student, error) {
    var s model.Student
    err := r.pool.QueryRow(ctx,
        "SELECT id, user_id, nim, nama, prodi, angkatan, ipk_terakhir, created_at FROM students WHERE id = $1 AND deleted_at IS NULL", id,
    ).Scan(&s.ID, &s.UserID, &s.NIM, &s>Nama, &s.Prodi, &s.Angkatan, &s.IPKTerakhir, &s.CreatedAt)
    if err != nil {
```

3.8 Centralized Error Handling dan Hashing (Bcrypt)

Semua password disimpan dalam bentuk hash menggunakan algoritma Bcrypt (cost = 10).

Pada pembuatan mahasiswa baru (POST /api/v1/students), sistem membuat record users (dengan password awal berupa NIM yang di-hash) dan record students dalam satu transaksi database.

Untuk standarisasi response error, sistem menerapkan Centralized Error Wrapper:

1. Kesalahan validasi input serta duplikasi nim atau email pada manajemen mahasiswa dikembalikan sebagai 422 Unprocessable Entity.
2. Duplikasi pengambilan mata kuliah yang sama di tahun akademik yang sama pada KRS dikembalikan sebagai 409 Conflict.
3. Kesalahan tak terduga pada server dikembalikan sebagai 500 Internal Server Error dengan pesan aman tanpa membocorkan stack trace di mode production.

Bukti Potongan Kode Bcrypt Hashing & Centralized Error Handler:

```
// HashPassword mengubah password menjadi hash yang tidak dapat dikembalikan.
func HashPassword(plain string) (string, error) {
    hashed, err := bcrypt.GenerateFromPassword([]byte(plain), bcryptCost)
    if err != nil {
        return "", err
    }
    return string(hashed), nil
}
```

```
// ErrorResponsePayload for standardized error output
func ErrorResponsePayload(c *fiber.Ctx, status int, message string, errors any) error {
    resp := model.WebResponse{
        Success: false,
        Message: message,
        Errors:  errors,
    }
    return c.Status(status).JSON(resp)
}
```

BAB IV

IMPLEMENTASI DAN PENGUJIAN 10 ENDPOINT TERPADU (UAT)

Bab ini menggabungkan spesifikasi teknis dari ke-10 endpoint beserta skenario pengujian User Acceptance Testing (UAT) yang mencakup Positive Testing (skenario sukses) dan Negative Testing (skenario validasi, otorisasi, dan batasan bisnis) secara langsung pada masing-masing endpoint.

4.1 Standarisasi Format Response JSON dan Ringkasan 10 Endpoint

Semua response API menggunakan struktur JSON yang seragam. Status code yang wajib digunakan mencakup: 200, 201, 204, 401, 403, 404, 409, 422, 429, dan 500.

Tabel Ringkasan 10 Endpoint:

No	Method	Endpoint	Akses	Fungsi	Status Sukses
1	POST	/api/v1/auth/login	Publik	Login, mengembalikan access token	200
2	GET	/api/v1/auth/me	Semua role	Profil pengguna yang sedang login	200
3	GET	/api/v1/students	Admin	Daftar mahasiswa dengan pagination, filter, search	200
4	POST	/api/v1/students	Admin	Menambah mahasiswa sekaligus akun user-nya	201
5	GET	/api/v1/students/{id}	Admin, Mahasiswa (data sendiri)	Detail mahasiswa beserta total SKS diambil	200
6	PUT	/api/v1/students/{id}	Admin	Memperbarui data mahasiswa	200

No	Method	Endpoint	Akses	Fungsi	Status Sukses
7	DELETE	/api/v1/students/{id}	Admin	Soft delete mahasiswa	204
8	GET	/api/v1/courses	Semua role	Daftar mata kuliah beserta sisa kuota	200
9	POST	/api/v1/enrollments	Mahasiswa	Mengambil mata kuliah (menambah KRS)	201
10	DELETE	/api/v1/enrollments/{id}	Mahasiswa (milik sendiri)	Membatalkan mata kuliah dari KRS	204

4.2 Tabel Ringkasan Matriks Skenario Pengujian (UAT)

ID	Endpoint Terkait	Skenario Pengujian (Test Case)	Jenis Uji	Ekspektasi Status	Status
TC-01	POST /api/v1/auth/login	Login dengan kredensial benar	Positive	200 OK	PASSED
TC-02	POST /api/v1/auth/login	Login dengan password salah	Negative	401 Unauthorized	PASSED
TC-03	POST /api/v1/auth/login	Login dengan format email tidak valid / password < 8 karakter	Negative	422 Unprocessable Entity	PASSED
TC-04	POST /api/v1/auth/login	Percobaan login gagal lebih dari 5 kali dalam 1 menit (Rate Limit)	Negative	429 Too Many Requests	PASSED
TC-05	GET /api/v1/auth/me	Mengambil profil pengguna yang sedang login dengan token valid	Positive	200 OK	PASSED

ID	Endpoint Terkait	Skenario Pengujian (Test Case)	Jenis Uji	Ekspektasi Status	Status
TC-06	GET /api/v1/auth/me	Mengakses profil tanpa token atau dengan token tidak valid/kedaluwarsa	Negative	401 Unauthorized	PASSED
TC-07	GET /api/v1/students	Admin mengambil daftar mahasiswa (pagination, filter, search, sort)	Positive	200 OK	PASSED
TC-08	GET /api/v1/students	Mahasiswa mencoba mengakses daftar seluruh mahasiswa (RBAC)	Negative	403 Forbidden	PASSED
TC-09	POST /api/v1/students	Admin menambah mahasiswa baru dengan data valid (Transaction)	Positive	201 Created	PASSED
TC-10	POST /api/v1/students	Admin menambah mahasiswa dengan NIM 11 digit atau NIM/Email duplikat	Negative	422 Unprocessable Entity	PASSED
TC-11	GET /api/v1/students/{id}	Mahasiswa melihat detail profil dan KRS miliknya sendiri	Positive	200 OK	PASSED
TC-12	GET /api/v1/students/{id}	Mahasiswa mencoba mengakses detail mahasiswa lain (Ownership)	Negative	403 Forbidden	PASSED
TC-13	PUT /api/v1/students/{id}	Admin memperbarui data mahasiswa aktif	Positive	200 OK	PASSED
TC-14	DELETE /api/v1/students/{id}	Admin melakukan Soft-Delete mahasiswa	Positive	204 No Content	PASSED

ID	Endpoint Terkait	Skenario Pengujian (Test Case)	Jenis Uji	Ekspektasi Status	Status
TC-15	DELETE /api/v1/students/{id}	Mahasiswa yang sudah di-soft-delete mencoba login / diakses detailnya	Negative	401 / 404 Not Found	PASSED
TC-16	GET /api/v1/courses	Mengambil daftar mata kuliah beserta perhitungan terisi dan sisa_kuota	Positive	200 OK	PASSED
TC-17	POST /api/v1/enrollments	Mahasiswa mengambil mata kuliah baru yang memenuhi syarat KRS	Positive	201 Created	PASSED
TC-18	POST /api/v1/enrollments	Mahasiswa mengambil mata kuliah yang kuotanya penuh (sisa_kuota = 0)	Negative	422 Unprocessable Entity	PASSED
TC-19	POST /api/v1/enrollments	Mahasiswa mengambil mata kuliah melebihi batas SKS sesuai IPK	Negative	422 Unprocessable Entity	PASSED
TC-20	POST /api/v1/enrollments	Mahasiswa mengambil mata kuliah yang sama dua kali di tahun akademik sama	Negative	409 Conflict	PASSED
TC-21	POST /api/v1/enrollments	Admin mencoba mengambil KRS (RBAC)	Negative	403 Forbidden	PASSED
TC-22	DELETE /api/v1/enrollments/{id}	Mahasiswa membatalkan mata kuliah dari KRS miliknya sendiri	Positive	204 No Content	PASSED
TC-23	DELETE /api/v1/enrollments/{id}	Mahasiswa mencoba membatalkan KRS milik mahasiswa lain / ID tidak ada	Negative	403 / 404 Not Found	PASSED

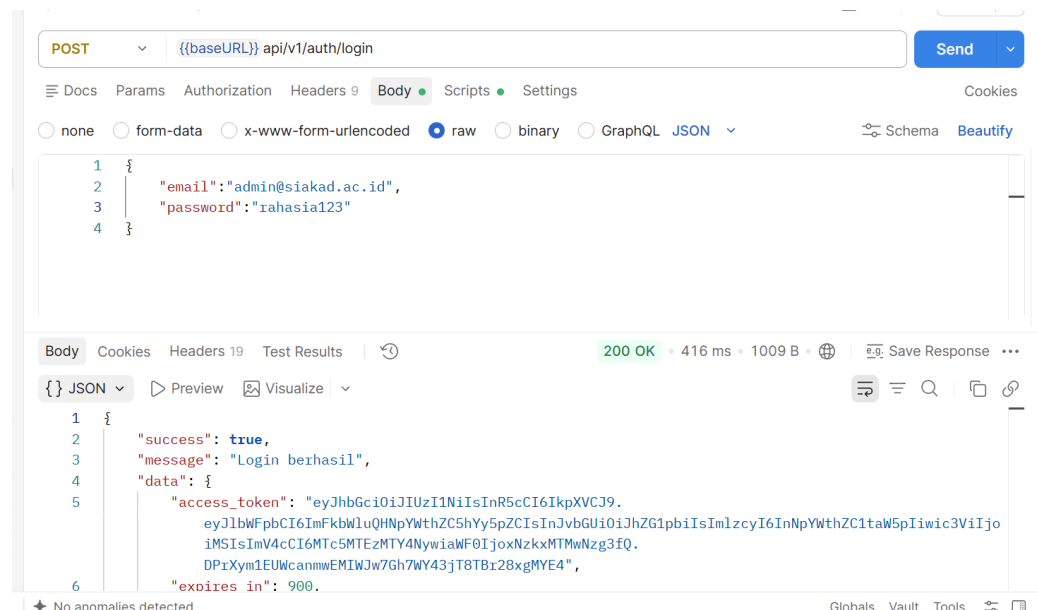
4.3 Rincian Spesifikasi dan Bukti Pengujian per Endpoint (Positive dan Negative Test)

4.3.1 Endpoint 1: POST /api/v1/auth/login

1. Akses: Publik
2. Request body: email (wajib, format email), password (wajib, min. 8 karakter).
3. Response Sukses (200 OK): access_token, token_type, expires_in, dan data user (id, email, role).
4. Aturan Error: 401 jika kredensial salah; 422 jika validasi gagal; 429 jika gagal login lebih dari 5 kali per menit (rate limiting).

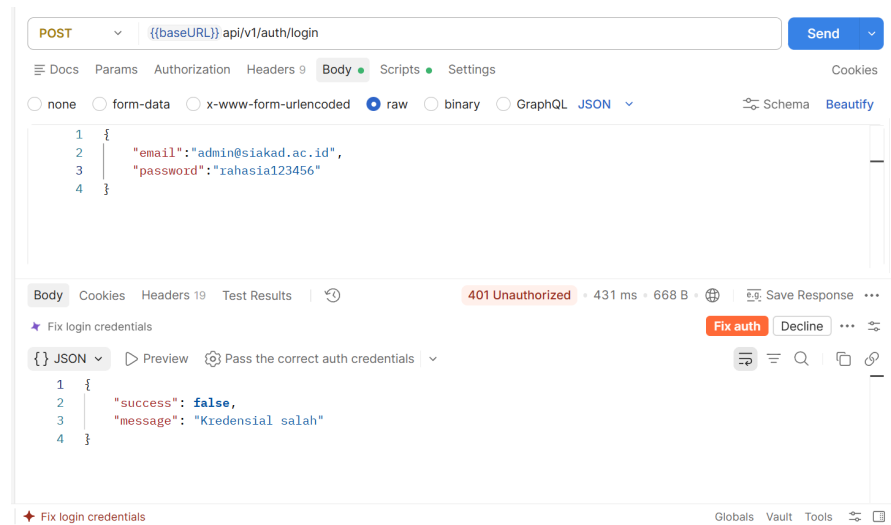
Skenario Pengujian Endpoint 1:

1. TC-01: Login dengan Kredensial Benar (Positive Test - 200 OK)
 - Input: Mengirimkan email dan password yang terdaftar dan aktif.
 - Ekspektasi: Status 200 OK, mengembalikan access_token JWT dan informasi user.
 - Bukti Pengujian Postman (TC-01):



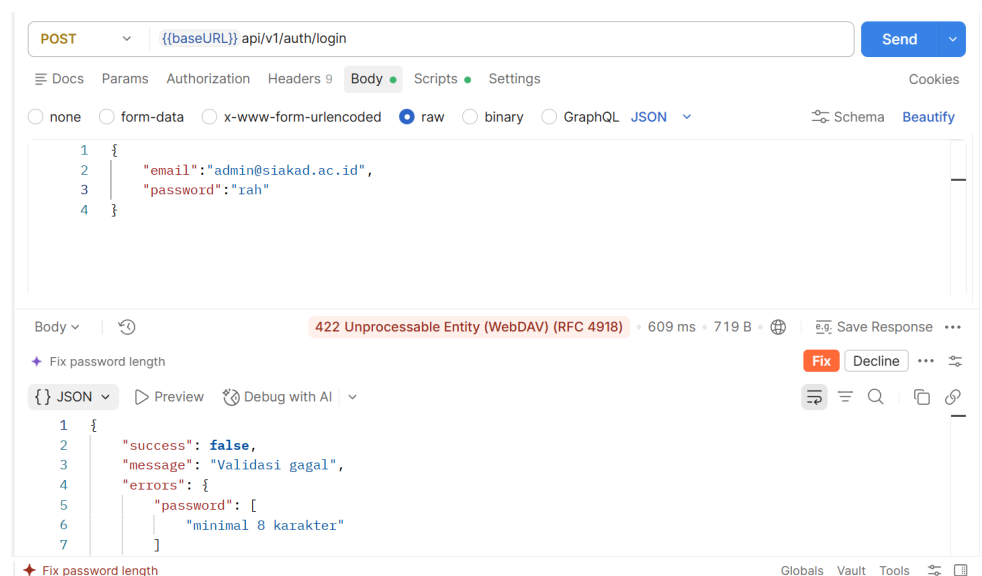
2. TC-02: Login dengan Password Salah (Negative Test - 401 Unauthorized)
 - Input: Mengirimkan email benar namun password salah.
 - Ekspektasi: Status 401 Unauthorized dengan pesan kredensial salah

- Bukti Pengujian Postman (TC-02):



3. TC-03: Login dengan Format Email Tidak Valid / Password Kurang dari 8 Karakter (Negative Test - 422 Unprocessable Entity)

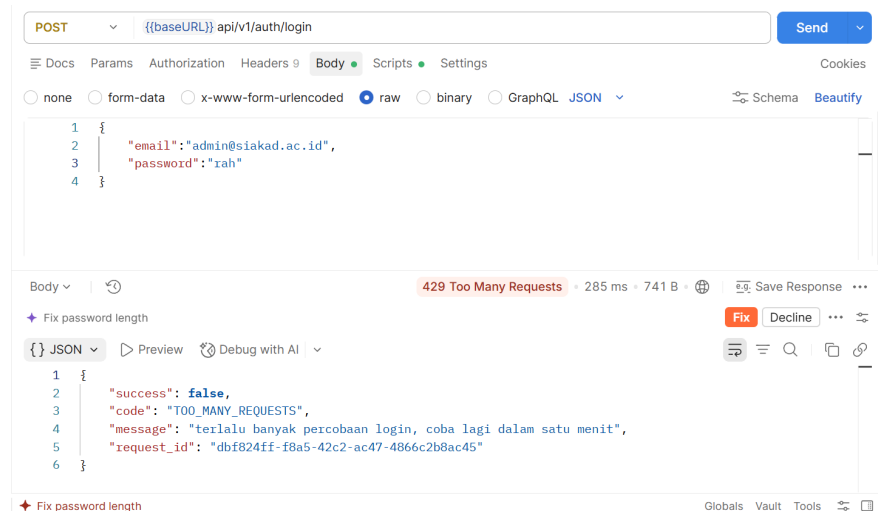
- Input: Mengirimkan email tanpa format standar atau password kurang dari 8 karakter.
- Ekspektasi: Status 422 Unprocessable Entity dengan detail validasi pada objek errors.
- Bukti Pengujian Postman (TC-03):



4. TC-04: Proteksi Brute-Force / Rate Limiting Login (Negative Test - 429 Too Many Requests)

- Input: Melakukan percobaan login gagal lebih dari 5 kali berturut-turut dalam 1 menit.
- Ekspektasi: Status 429 Too Many Requests.

- **Bukti Pengujian Postman (TC-04):**



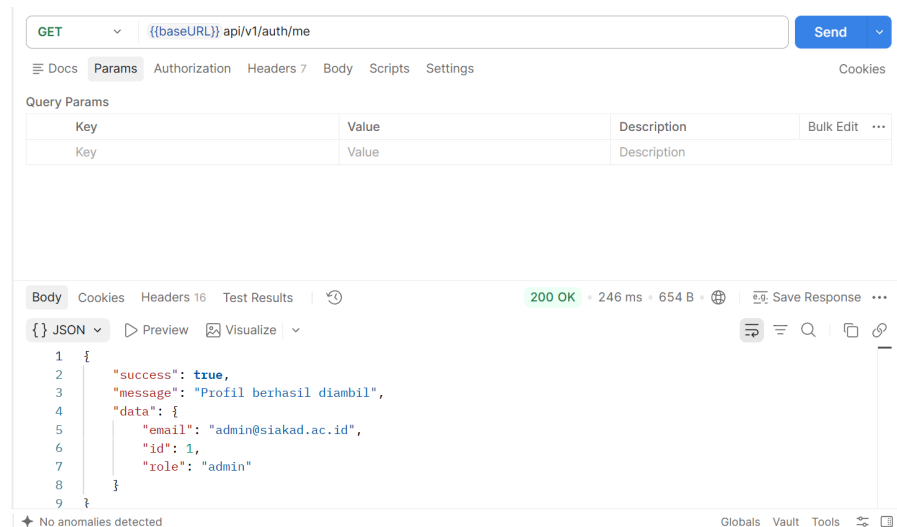
4.3.2 Endpoint 2: GET /api/v1/auth/me

1. Akses: Semua role (admin dan mahasiswa)
2. Header: Authorization: Bearer <token>
3. Response Sukses (200 OK): Data user; jika role mahasiswa, menyertakan objek students (nim, nama, prodi, angkatan).
4. Aturan Error: 401 jika token tidak ada, salah, atau kedaluwarsa.

Skenario Pengujian Endpoint 2:

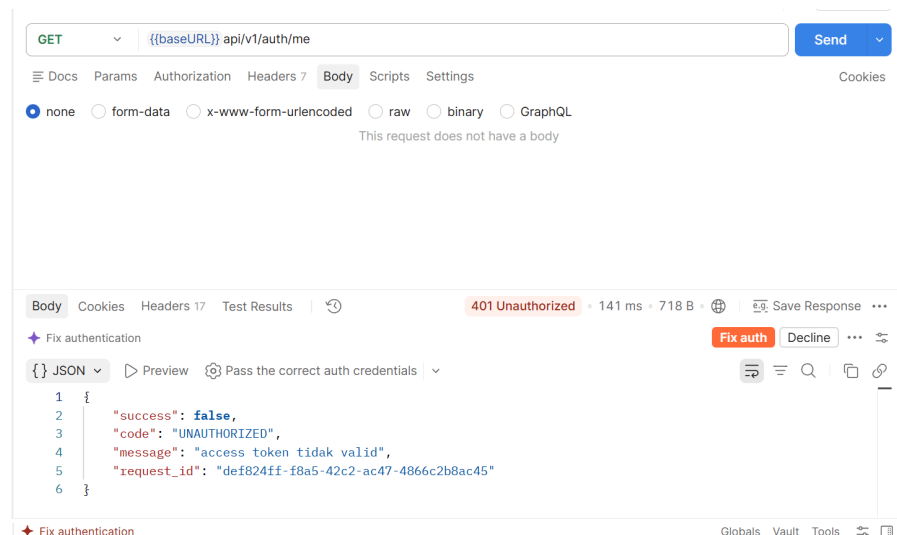
1. TC-05: Mengambil Profil Pengguna yang Sedang Login (Positive Test - 200 OK)
 - Input: Mengirimkan request dengan Header Authorization: Bearer <token_mahasiswa>.
 - Ekspektasi: Status 200 OK, mengembalikan data user beserta detail objek student.

- **Bukti Pengujian Postman (TC-05):**



2. TC-06: Mengakses Profil Tanpa Token / Token Tidak Valid (Negative Test - 401 Unauthorized)

- **Input:** Mengirimkan request tanpa header Authorization atau dengan token yang sudah kedaluwarsa/salah.
- **Ekspektasi:** Status 401 Unauthorized.
- **Bukti Pengujian Postman (TC-06):**



4.3.3 Endpoint 3: GET /api/v1/students

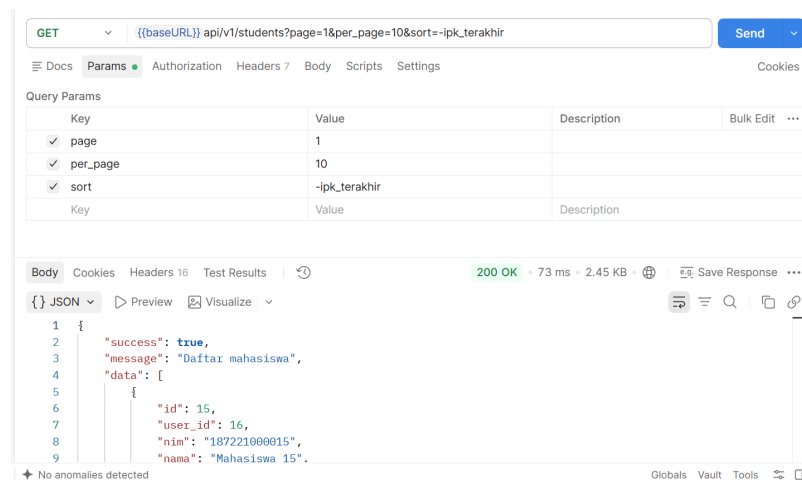
1. **Akses:** Admin
2. **Query parameter:** page (default 1), per_page (default 10, maks. 50), prodi, angkatan, search (mencari nim atau nama), sort (nama atau -ipk_terakhir).
3. **Response Sukses (200 OK):** Array data mahasiswa + objek meta (current_page, per_page, total, last_page).

4. Aturan Error: 401 jika tidak terautentikasi; 403 jika diakses oleh selain admin.

Skenario Pengujian Endpoint 3:

1. TC-07: Admin Mengambil Daftar Mahasiswa dengan Pagination dan Filter (Positive Test - 200 OK)

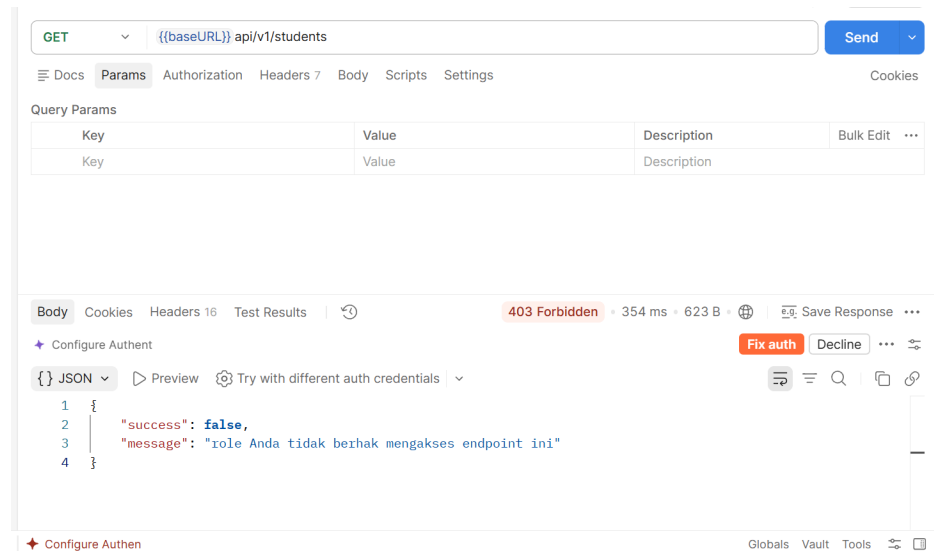
- Input: Admin memanggil GET
`/api/v1/students?page=1&per_page=10&sort=-ipk_terakhir`
- Ekspektasi: Status 200 OK, menampilkan daftar mahasiswa aktif beserta metadata paginasi.
- Bukti Pengujian Postman (TC-07):



2. TC-08: Mahasiswa Mencoba Mengakses Daftar Seluruh Mahasiswa (Negative Test - 403 Forbidden)

- Input: Mahasiswa memanggil GET `/api/v1/students` menggunakan token ber-role mahasiswa.
- Ekspektasi: Status 403 Forbidden (ditolak oleh middleware RBAC).

- **Bukti Pengujian Postman (TC-08):**



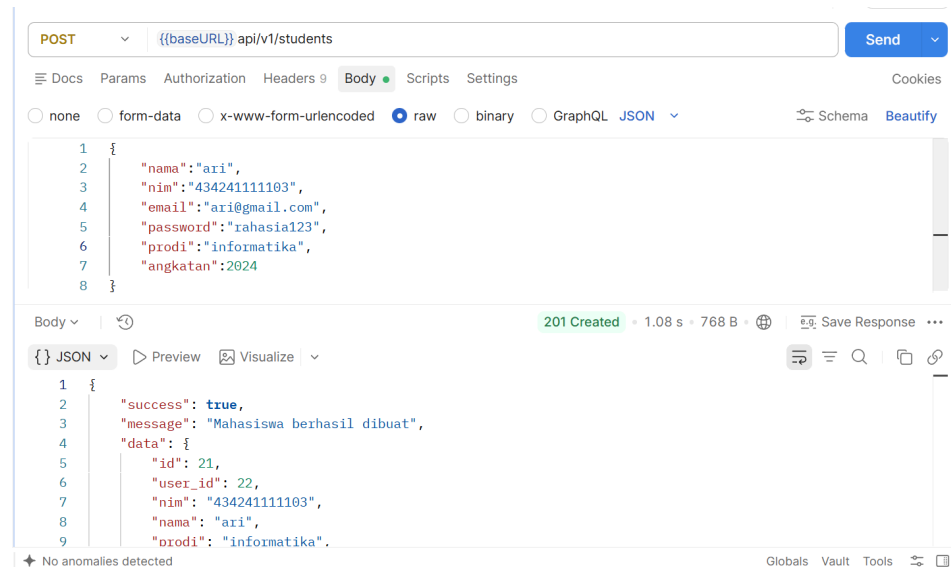
4.3.4 Endpoint 4: POST /api/v1/students

1. Akses: Admin
2. Request body: nim (wajib, unik, 12 digit), nama (wajib), email (wajib, unik), prodi (wajib), angkatan (wajib, 4 digit, <= tahun berjalan), ipk_terakhir (opsional, 0,00–4,00).
3. Proses: Membuat record users (role mahasiswa, password awal = nim yang di-hash) dan students dalam satu database transaction.
4. Response Sukses: 201 Created
5. Aturan Error: 422 jika nim/email duplikat atau validasi gagal; 403 jika bukan admin.

Skenario Pengujian Endpoint 4:

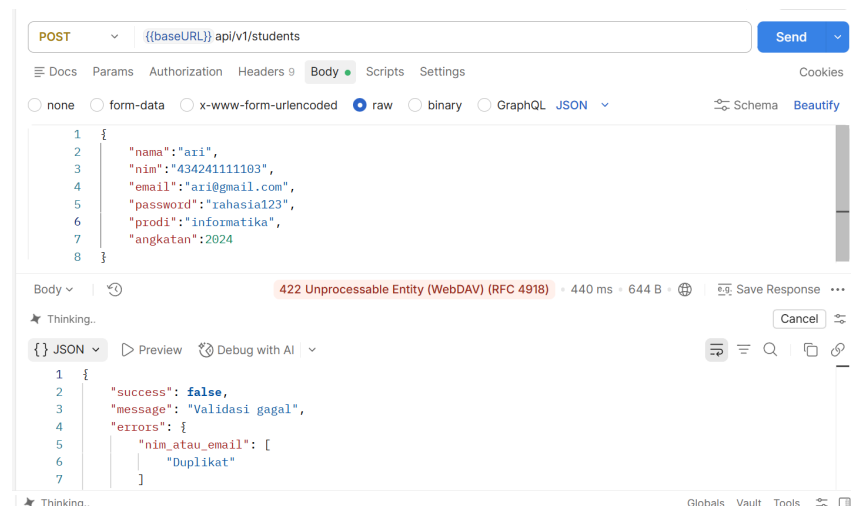
1. TC-09: Admin Menambah Mahasiswa Baru dengan Data Valid (Positive Test - 201 Created)
 - Input: Admin mengirimkan payload JSON mahasiswa baru sesuai aturan validasi.
 - Ekspektasi: Status 201 Created, akun users dan data students terbentuk dalam satu transaksi.

- **Bukti Pengujian Postman (TC-09):**



2. TC-10: Admin Menambah Mahasiswa dengan NIM Tidak Valid / NIM dan Email Duplikat (Negative Test - 422 Unprocessable Entity)

- Input: Admin mengirimkan nim 11 digit atau menggunakan nim/email yang sudah terdaftar.
- Ekspektasi: Status 422 Unprocessable Entity beserta rincian pesan kesalahan pada objek errors.
- **Bukti Pengujian Postman (TC-10):**



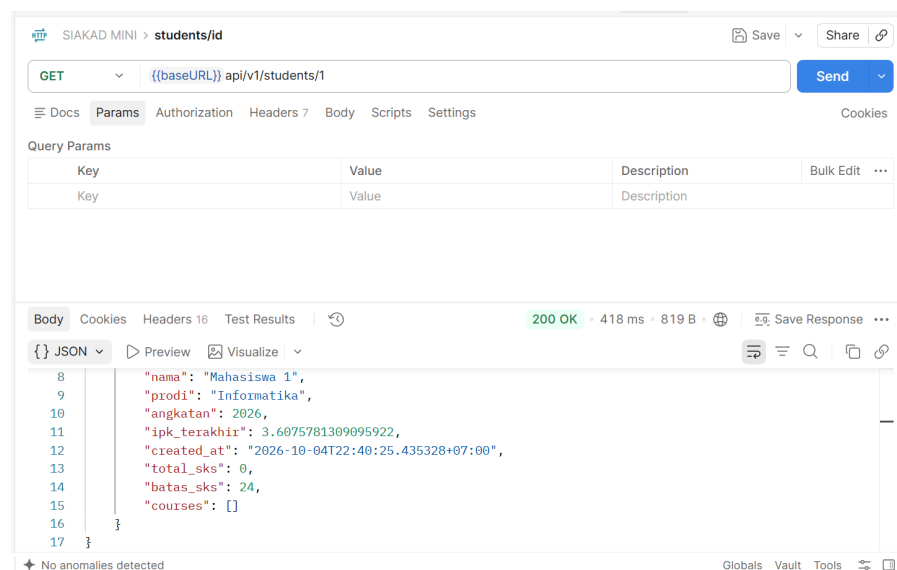
4.3.5 Endpoint 5: GET /api/v1/students/{id}

1. Akses: Admin, Mahasiswa (hanya data sendiri)
2. Response Sukses (200 OK): Data mahasiswa + daftar mata kuliah yang diambil + total_sks + batas_sks.

- Aturan Error: 403 jika mahasiswa mengakses data mahasiswa lain; 404 jika tidak ditemukan atau sudah di-soft delete.

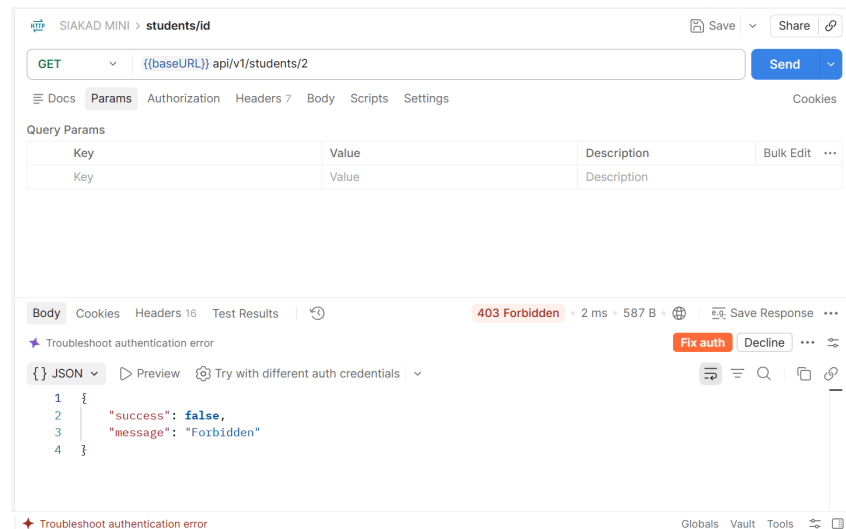
Skenario Pengujian Endpoint 5:

- TC-11: Mahasiswa Melihat Detail Profil dan KRS Miliknya Sendiri (Positive Test - 200 OK)
 - Input: Mahasiswa memanggil GET `/api/v1/students/{id}` sesuai dengan ID miliknya sendiri (atau diakses oleh Admin).
 - Ekspektasi: Status 200 OK, mengembalikan profil mahasiswa, daftar MK di KRS, `total_sks`, dan `batas_sks`.
 - Bukti Pengujian Postman (TC-11):



- TC-12: Mahasiswa Mengakses Detail Mahasiswa Lain (Negative Test - 403 Forbidden)
 - Input: Mahasiswa dengan `student_id = 1` mencoba mengakses GET `/api/v1/students/2`.
 - Ekspektasi: Status 403 Forbidden karena melanggar kepemilikan data.

- **Bukti Pengujian Postman (TC-12):**



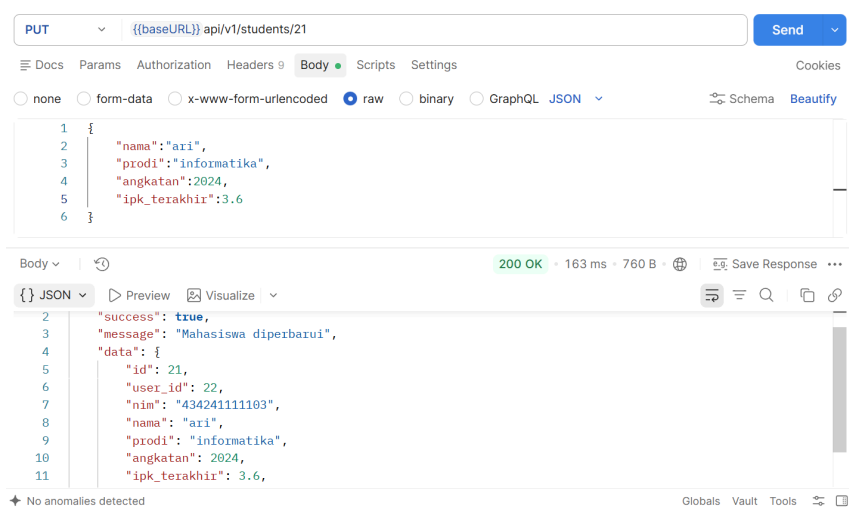
4.3.6 Endpoint 6: PUT /api/v1/students/{id}

1. Akses: Admin
2. Request body: nama, prodi, angkatan, ipk_terakhir (field yang sama dengan endpoint 4, kecuali nim tidak boleh diubah).
3. Response Sukses: 200 OK
4. Aturan Error: 404 jika tidak ditemukan; 422 jika validasi gagal; 403 jika bukan admin.

Skenario Pengujian Endpoint 6:

1. TC-13: Admin Memperbarui Data Mahasiswa (Positive Test - 200 OK)
 - Input: Admin mengirimkan perubahan data nama, prodi, angkatan, dan ipk_terakhir yang valid.
 - Ekspektasi: Status 200 OK beserta data mahasiswa terbaru yang telah diperbarui.

- **Bukti Pengujian Postman (TC-13):**



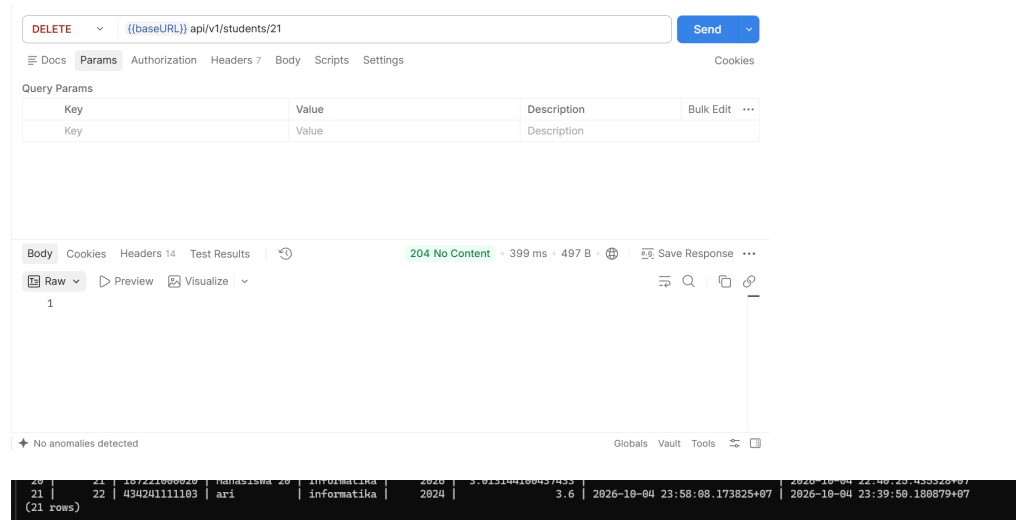
4.3.7 Endpoint 7: DELETE /api/v1/students/{id}

1. Akses: Admin
2. Proses: Soft delete (mengisi kolom `deleted_at`); mahasiswa yang dihapus tidak muncul di endpoint 3, mengembalikan 404 di endpoint 5, dan tidak dapat login.
3. Response Sukses: 204 No Content (Tanpa response body)
4. Aturan Error: 404 jika tidak ditemukan; 403 jika bukan admin.

Skenario Pengujian Endpoint 7:

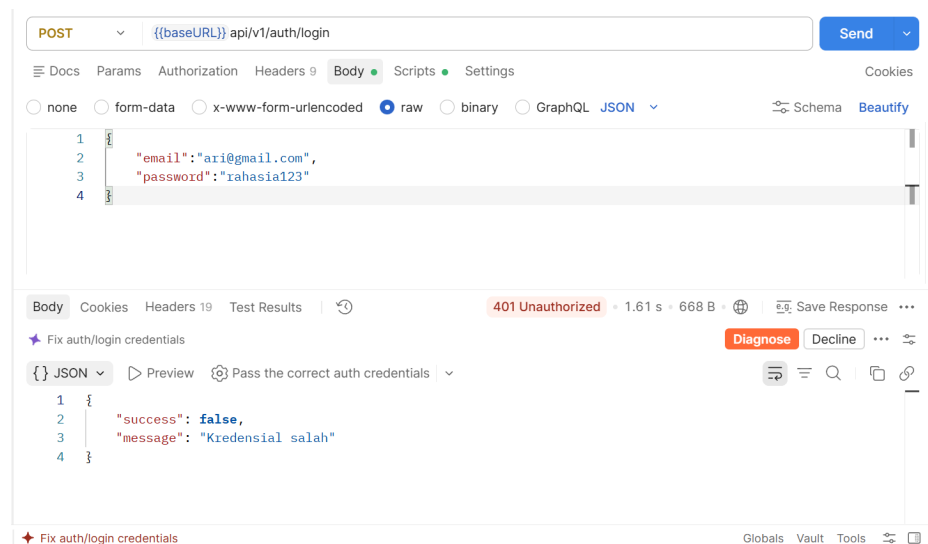
1. TC-14: Admin Melakukan Soft-Delete Mahasiswa (Positive Test - 204 No Content)
 - Input: Admin memanggil DELETE `/api/v1/students/{id}` pada mahasiswa aktif.
 - Ekspektasi: Status 204 No Content dan kolom `deleted_at` terisi di database.

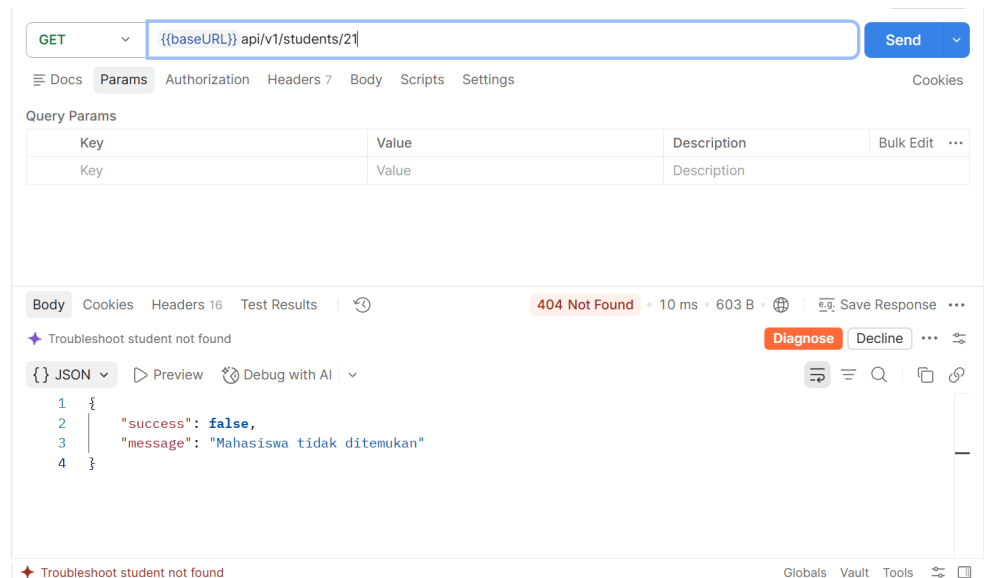
- **Bukti Pengujian Postman (TC-14):**



2. TC-15: Mahasiswa yang Sudah Di-Soft-Delete Mencoba Login / Diakses Detailnya (Negative Test - 401 dan 404)

- **Input:** Mencoba login menggunakan akun mahasiswa yang baru saja di-soft-delete atau memanggil GET /api/v1/students/{id}-nya.
- **Ekspektasi:** Login ditolak dengan 401 Unauthorized dan akses detail mengembalikan 404 Not Found.
- **Bukti Pengujian Postman (TC-15):**





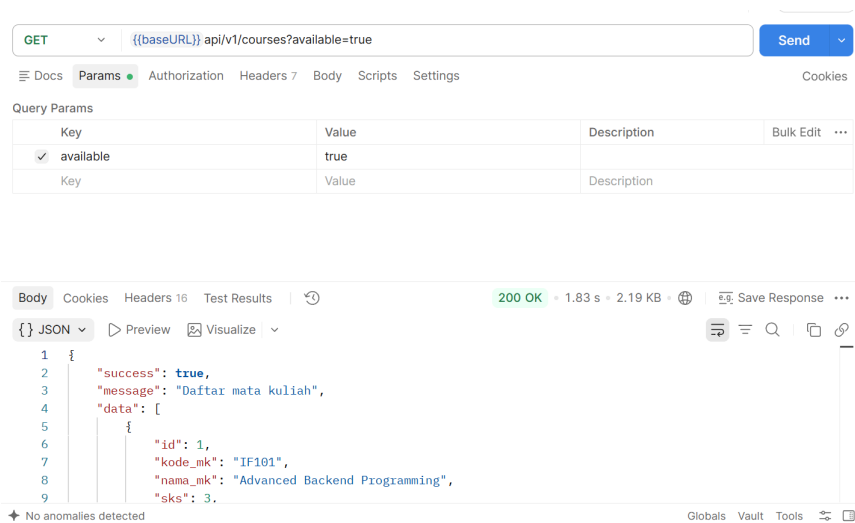
4.3.8 Endpoint 8: GET /api/v1/courses

1. Akses: Semua role (admin dan mahasiswa)
2. Query parameter: semester, search (kode_mk atau nama_mk), available=true (hanya mata kuliah yang kuotanya belum penuh).
3. Response Sukses (200 OK): Daftar mata kuliah, di mana tiap item menyertakan terisi dan sisa_kuota yang dihitung dari tabel enrollments.
4. Aturan Error: 401 jika tidak terautentikasi.

Skenario Pengujian Endpoint 8:

1. TC-16: Mengambil Daftar Mata Kuliah Beserta Perhitungan Sisa Kuota (Positive Test - 200 OK)
 - Input: Memanggil GET /api/v1/courses?available=true dengan token valid.
 - Ekspektasi: Status 200 OK, menampilkan daftar mata kuliah beserta nilai terisi dan sisa_kuota secara akurat.

- **Bukti Pengujian Postman (TC-16):**



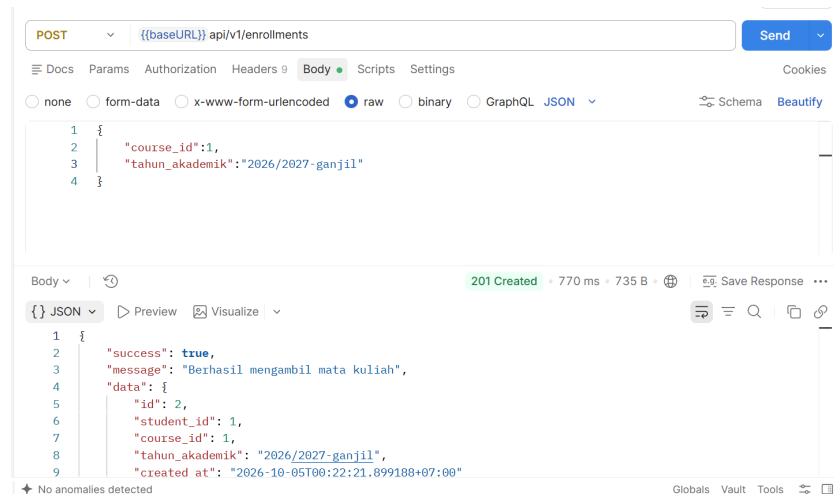
4.3.9 Endpoint 9: POST /api/v1/enrollments

1. Akses: Mahasiswa
2. Request body: `course_id` (wajib, harus ada), `tahun_akademik` (wajib, format 2026/2027-Ganjil).
3. Proses: Dalam satu database transaction, mengecek duplikasi, ketersediaan kuota (dengan row locking `SELECT ... FOR UPDATE`), dan batas SKS sesuai business rule IPK.
4. Response Sukses: 201 Created
5. Aturan Error: 409 jika sudah pernah diambil pada tahun akademik yang sama; 422 jika kuota penuh atau total SKS melebihi batas (menyertakan pesan yang menyebut sisa SKS); 403 jika bukan mahasiswa.

Skenario Pengujian Endpoint 9:

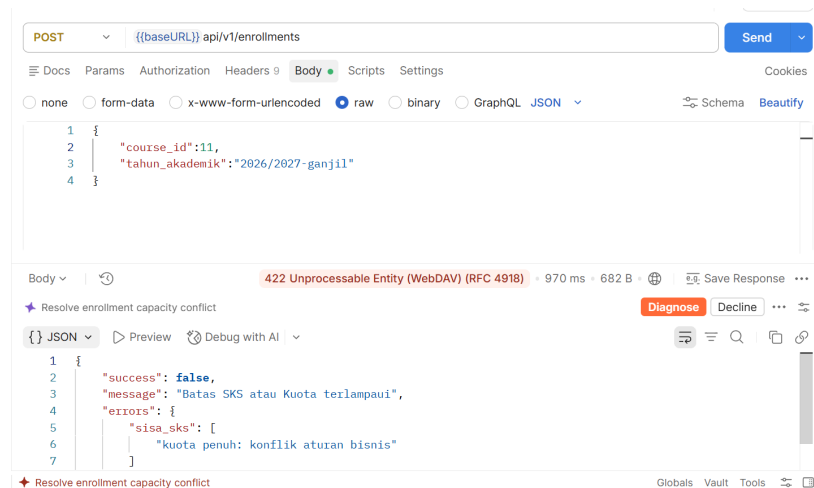
1. TC-17: Mahasiswa Mengambil Mata Kuliah Baru yang Memenuhi Syarat KRS (Positive Test - 201 Created)
 - Input: Mahasiswa mengirimkan `course_id` yang kuotanya masih tersedia dan `tahun_akademik` valid, serta total SKS masih mencukupi.
 - Ekspektasi: Status 201 Created, KRS berhasil ditambahkan dan `sisa_kuota` berkurang 1.

- **Bukti Pengujian Postman (TC-17):**



2. **TC-18: Mahasiswa Mengambil Mata Kuliah yang Kuotanya Habis (Negative Test - 422 Unprocessable Entity)**

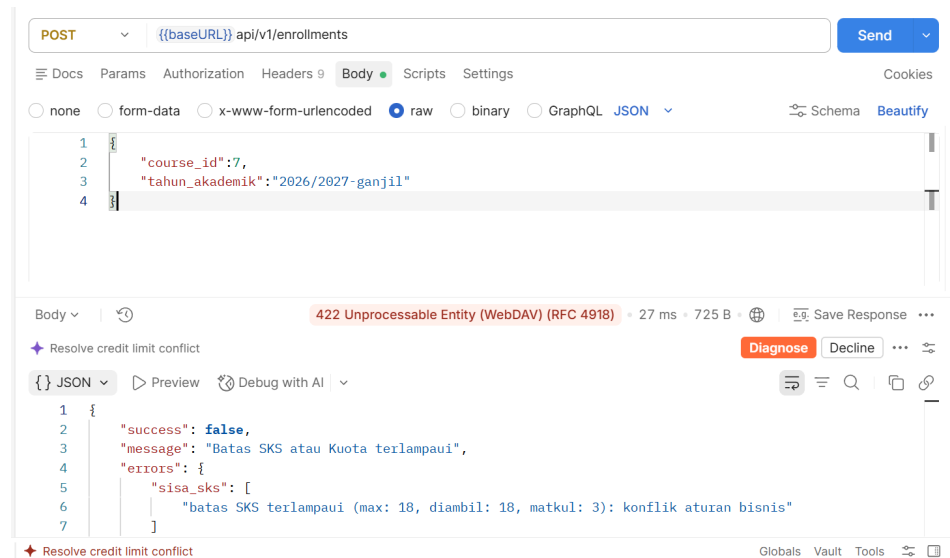
- **Input:** Mahasiswa mencoba mengambil `course_id` yang memiliki `sisa_kuota = 0`.
- **Ekspektasi:** Status 422 Unprocessable Entity dengan pesan bahwa kuota mata kuliah telah penuh.
- **Bukti Pengujian Postman (TC-18):**



3. **TC-19: Mahasiswa Mengambil Mata Kuliah Melebihi Batas SKS IPK (Negative Test - 422 Unprocessable Entity)**

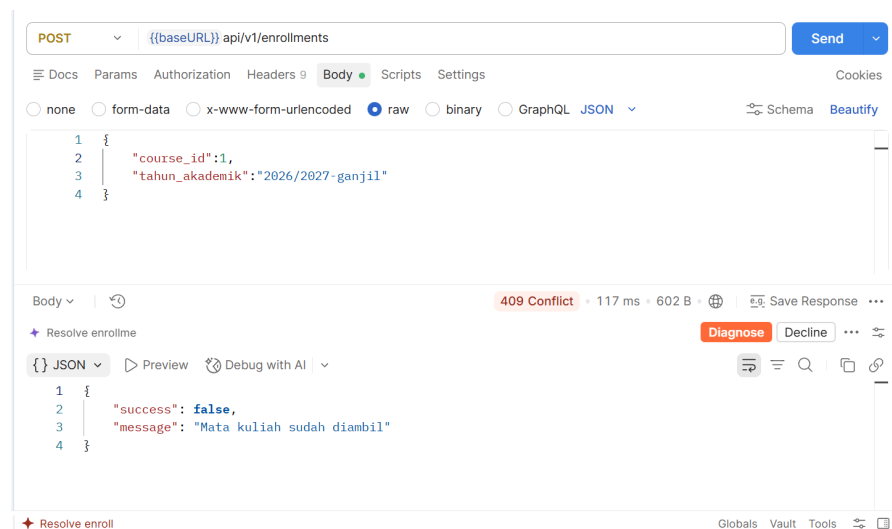
- **Input:** Mahasiswa dengan IPK 2,10 (batas maksimal 18 SKS) mengambil mata kuliah yang menyebabkan total SKS melampaui 18 SKS.
- **Ekspektasi:** Status 422 Unprocessable Entity disertai pesan yang menyebutkan sisa SKS yang masih dapat diambil.

- Bukti Pengujian Postman (TC-19):



4. TC-20: Mahasiswa Mengambil Mata Kuliah yang Sama Dua Kali di Tahun Akademik Sama (Negative Test - 409 Conflict)

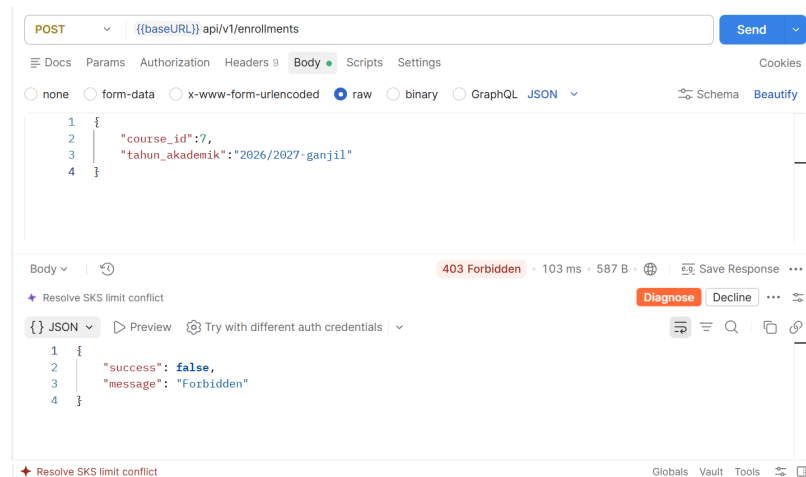
- Input: Mahasiswa mengirimkan kembali `course_id` dan `tahun_akademik` yang sudah diambil pada TC-17.
- Ekspektasi: Status 409 Conflict karena melanggar aturan duplikasi KRS.
- Bukti Pengujian Postman (TC-20):



5. TC-21: Admin Mencoba Mengambil KRS (Negative Test - 403 Forbidden)

- Input: Pengguna ber-role admin mencoba memanggil `POST /api/v1/enrollments`.
- Ekspektasi: Status 403 Forbidden karena pengambilan KRS hanya diizinkan bagi mahasiswa.

- **Bukti Pengujian Postman (TC-21):**



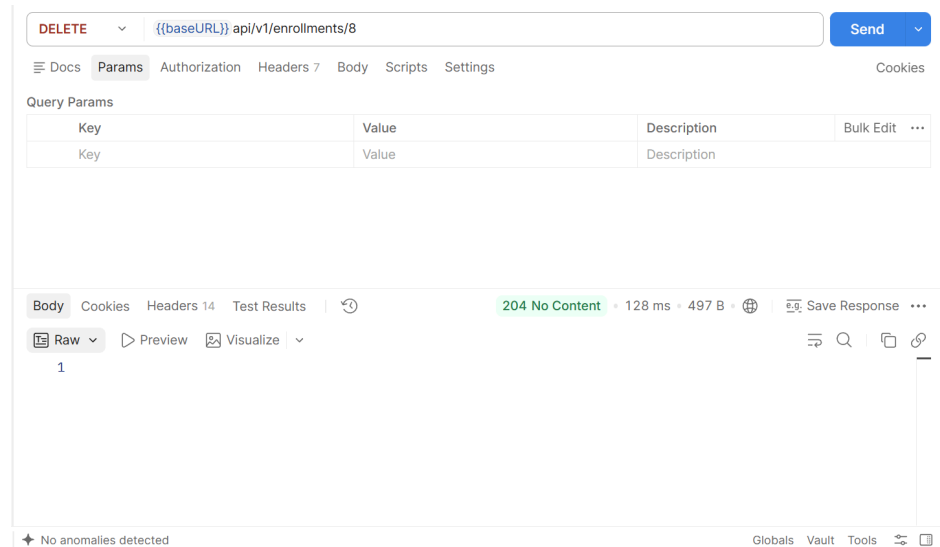
4.3.10 Endpoint 10: DELETE /api/v1/enrollments/{id}

1. Akses: Mahasiswa (hanya milik sendiri)
2. Proses: Menghapus record enrollments milik mahasiswa yang sedang login; kuota mata kuliah kembali bertambah secara otomatis.
3. Response Sukses: 204 No Content (Tanpa response body)
4. Aturan Error: 403 jika enrollment milik mahasiswa lain; 404 jika tidak ditemukan.

Skenario Pengujian Endpoint 10:

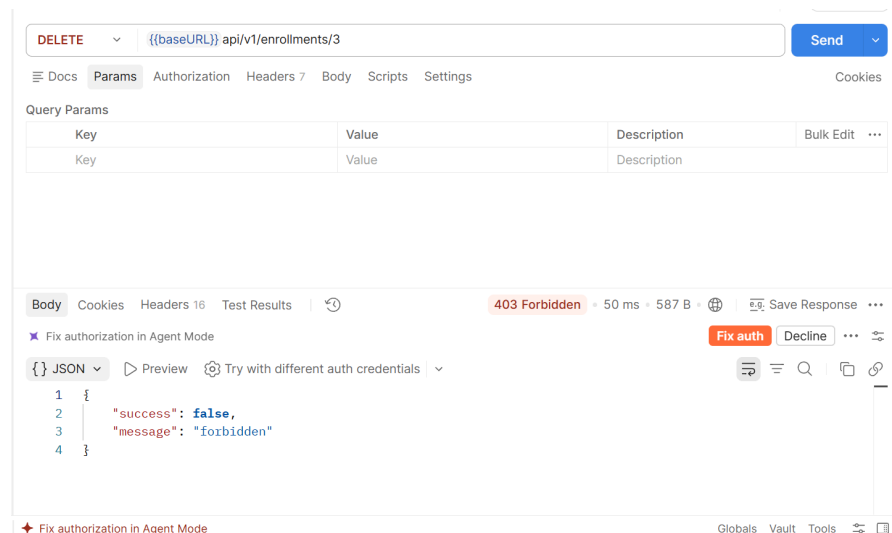
1. TC-22: Mahasiswa Membatalkan KRS Miliknya Sendiri (Positive Test - 204 No Content)
 - Input: Mahasiswa memanggil DELETE `/api/v1/enrollments/{id}` menggunakan ID enrollment miliknya sendiri.
 - Ekspektasi: Status 204 No Content, record KRS terhapus, dan sisa_kuota mata kuliah bertambah kembali.

- **Bukti Pengujian Postman (TC-22):**



2. TC-23: Mahasiswa Mencoba Membatalkan KRS Milik Mahasiswa Lain / ID Tidak Ditemukan (Negative Test - 403 dan 404)

- **Input:** Mahasiswa mencoba menghapus ID enrollment milik mahasiswa lain (menghasilkan 403 Forbidden) atau ID yang tidak terdaftar (menghasilkan 404 Not Found).
- **Ekspektasi:** Status 403 Forbidden atau 404 Not Found.
- **Bukti Pengujian Postman (TC-23):**



DELETE

{{baseUrl}} api/v1/enrollments/9999

Send

Docs

Params

Authorization

Headers 7

Body

Scripts

Settings

Cookies

Query Params

Key	Value	Description	Bulk Edit	...
Key	Value	Description		

Body

Cookies

Headers 16

Test Results

🕒

404 Not Found

15 ms

597 B

🌐

Save Response

...

Fix enrollment ID

Diagnose

Decline

...

🔍

🔗

JSON

Preview

🐞 Debug with AI

▼

```
1 {
2   "success": false,
3   "message": "KRS tidak ditemukan"
4 }
```

Fix enrollment ID

Globals

Vault

Tools

🔍

📄

BAB V

KESIMPULAN DAN SARAN PENGEMBANGAN

5.1 Kesimpulan

Proyek RESTful API SIAKAD Mini telah berhasil diimplementasikan secara menyeluruh menggunakan bahasa pemrograman Go, framework Go Fiber, dan database PostgreSQL sesuai dengan seluruh spesifikasi teknis dan aturan bisnis UTS. Seluruh 10 endpoint beserta 23 skenario pengujian terpadu (Positive dan Negative Testing) telah berfungsi dengan akurat, dilengkapi mekanisme keamanan JWT, RBAC, Bcrypt hashing, Rate Limiting, serta pengelolaan transaksi database (SELECT ... FOR UPDATE) yang menjamin konsistensi kuota mata kuliah dan batas SKS mahasiswa.